

AI-Based Civic Problem Reporter

Smart City Issue Detection, Severity Estimation & Department Routing

Project Proposal for UCS503P

Submitted to: Dr Nisha Thakur

Thapar Institute of Engineering and Technology

Chintan Sood , CSED

Tanu Bansal , CSED

Adhyan Gupta , CSED

August 17, 2026

1 Higher-order goal

This project aims to improve citizen-government responsiveness in urban India by reducing the time, effort, and ambiguity involved in reporting and resolving civic infrastructure issues. While the scope is broader than any single municipality, the immediate engineering objective is to build a deployable platform that uses computer vision to automate the detection, prioritization, and routing of civic issues — translating raw citizen photo uploads into structured, actionable reports for municipal departments.

2 Time-to-value

- We will validate the core ML pipeline (YOLOv8 fine-tuned on civic-issue datasets) within the first two weeks, before building any application layer, to confirm feasibility early.
- We will deliver working vertical slices per sprint — each person owns a complete feature end-to-end (ML + backend + frontend) rather than one layer of the entire system.
- Success in each iteration is defined by a demonstrable, working pipeline increment: photo in, structured detection result out, visible on a live map.
- We will use CI/CD to enable safe, frequent releases to a staging environment from early in the project.

3 Problem Statement

Citizens in Indian cities regularly encounter civic infrastructure issues — potholes, broken streetlights, overflowing garbage, water leakage, fallen trees, and damaged sidewalks. Reporting these issues today is manual, fragmented, and largely invisible to the citizen once submitted. Common pain points include:

- Fragmentation: complaints are made via multiple unconnected channels (calls,

Roll No. 1024030802 , Email: agupta30_be24@thapar.edu

Roll No. 1024030806 , Email: csood_be24@thapar.edu

Roll No. 1024030807 , Email: tbansal3_be24@thapar.edu

WhatsApp, social media, paper forms), causing lost context and duplicated effort.

Manual categorization: citizens must self-identify the issue type, leading to mislabeled or incomplete reports that require staff rework.

No severity signal: all reports are treated equally regardless of how dangerous or widespread the issue is, making prioritization entirely subjective.

- Duplicate overload: the same pothole may be reported dozens of times by different citizens, flooding staff queues with redundant entries.
- Slow routing: a general inbox requires a human to manually decide which department handles each report before any action can begin.
- Low transparency: citizens have no way to track what happened to their report after submission.

Why this matters now (contextual relevance): Smart-city and e-governance initiatives across India are increasing pressure on municipalities to digitize citizen services. A system that automates detection, triage, and routing — rather than just digitizing a paper form — meaningfully reduces the staff bandwidth required to process reports and increases citizen trust through visible status tracking.

4 Proposed Solution

4.1 Overview

Build a web-based AI-powered civic reporting platform with the following components:

- Citizen submission portal: users upload a photo of a civic issue; GPS location is autocaptured.
- ML detection pipeline: YOLOv8 (fine-tuned on labeled civic-issue datasets) automatically detects the issue type and estimates severity from the uploaded image.
- Duplicate detection: GPS proximity + lightweight image similarity comparison flags likely duplicates before a new report is created.
- Automated department routing: issue type is mapped to the responsible department via rule-based logic (e.g., pothole → Roads Department, garbage → Sanitation Department).
- Urgency scoring: a formula combining severity, time unresolved, and duplicate report count produces a priority score for department queues.
- Live city map: all active reports are plotted and color-coded by urgency/severity on a Leaflet map.
- Department dashboard: staff view a prioritized, filterable queue of issues assigned to their department with status management.
- Admin dashboard: city-level overview of all reports, resolution metrics, and department performance.

4.2 Core workflow

- Citizen uploads photo via the web app; GPS coordinates are captured automatically with a manual pin-correction fallback.
Image is compressed client-side and sent to the backend; stored in file storage with only the file path saved to the database.
YOLOv8 fine-tuned model runs inference: outputs issue type, confidence score, and bounding box coordinates.
- Severity is calculated from confidence score and bounding box area relative to image size.
- Duplicate check: new report's GPS coordinates and image embedding are compared against nearby existing reports; if matched, the report is merged and the duplicate count incremented.
- Department routing assigns the report to the correct department based on issue type.
- Urgency score is computed and the report is stored in PostgreSQL with all metadata.
- Report appears as a colored pin on the citizen-facing map and in the department staff queue, sorted by urgency.
- Department staff update status (Pending → In Progress → Resolved); citizen can track status on their dashboard.

4.3 Operational constraints for an educational, tier-2/3 setting

- The solution must run on free-tier or low-cost hosting (Render/Railway for backend, Vercel for frontend, Neon/Supabase for PostgreSQL).
- Image processing must remain responsive on a standard laptop GPU or free Colab instance; YOLOv8n or YOLOv8s variants are used rather than larger models.
- Client-side image compression ensures the system is usable on mid-range mobile devices with limited upload bandwidth.
- GPS dependency is handled gracefully: if permission is denied or GPS is unavailable, the citizen manually pins their location on a map.

5 Solution Approach ...Engineering focus

This is an engineering course project. The focus is on a complete, integrated, deployable system — not on novel algorithm research. Each component uses a well-established, documented approach.

5.1 ML detection pipeline

The core AI component is YOLOv8 fine-tuned on our own labeled dataset — not a thirdparty hosted AI API. The distinction matters: we download pretrained weights, run training on our own labeled civic-issue images using a free Colab GPU, and deploy the resulting best.pt weights on our own backend. The model is not called from an external service; it runs as part of our inference pipeline.

Roll No. 1024030802 , Email:agupta30_be24@thapar.edu

Roll No. 1024030806 , Email:csood_be24@thapar.edu

Roll No. 1024030807 , Email:tbansal3_be24@thapar.edu

- Pretrained base: YOLOv8s (small variant, balances speed and accuracy for our scale).
- Fine-tuning dataset: merged from public labeled datasets on Roboflow Universe (pothole detection, garbage detection, street damage) plus team-collected and annotated photos for underrepresented classes.
Training: run on Google Colab free GPU (T4); tracked across epochs by mAP50, box loss, and class loss; early stopping with patience=20.
Severity formula (our own logic on top of YOLO output): $\text{severity_score} = (\text{confidence} \times 0.6) + (\text{bbox_area} / \text{image_area} \times 0.4)$; thresholded into Low / Medium / High.
- Inference script: a clean `detect_issue(image_path)` function returns structured JSON (issue_type, confidence, bbox, severity) — the contract the backend calls.

5.2 Duplicate and urgency logic

We deliberately separate what is ML-driven from what is rule-based, and we document this distinction in our SRS and report. Duplicate detection and urgency scoring are not ML models — they are deterministic formulas applied on top of ML output:

- Duplicate detection: if a new report's GPS coordinates are within a configurable radius (default 30 m) of an existing open report of the same issue type, it is flagged as a likely duplicate for staff review. Image similarity (cosine similarity of lightweight embeddings) adds a secondary signal. Staff make the final decision; the system never auto-deletes.
- Urgency score: $\text{urgency} = (\text{severity_weight} \times 0.5) + (\text{duplicate_count} \times 0.3) + (\text{days_unresolved} \times 0.2)$, normalized to 0–100. This produces the sort order in the department queue.

5.3 Web architecture

- Frontend: React with Tailwind CSS; react-leaflet for map; recharts for admin analytics.
- Backend: Python FastAPI; background tasks for async ML inference so uploads do not block.
- Database: PostgreSQL (SQLAlchemy ORM); indexed on latitude, longitude, issue_type, status for duplicate and filter queries.
- File storage: uploaded images stored to disk (pilot) or cloud object storage (production); only file paths stored in DB.
- Authentication: JWT with role-based access control (citizen / department staff / admin).

5.4 CI/CD and fast delivery enabler

- GitHub Actions: run linting + backend unit tests + frontend build check on every push.

- Staging deployment on every merge to main via Render deploy hooks.
- Model weights versioned separately (Git LFS or Drive); inference script is stateless and independently testable.

6 Evaluation Criterion

6.1 Primary evaluation metric

Detection accuracy (mAP50): mean average precision at IoU threshold 0.5 on a held-out test set, reported per class (pothole, garbage, streetlight, etc.) and overall.

- Target: overall mAP50 ≥ 0.70 on the held-out test set.
- Attribution: measured from YOLOv8 validation run on the 15% test split withheld from training.

6.2 Secondary evaluation metrics

- Severity accuracy: manual review of 50 test detections to confirm severity label (Low/Medium/High) matches human judgment.
- Duplicate detection precision: percentage of flagged duplicates that are genuine duplicates on manual review (target $\geq 80\%$).
- End-to-end latency: time from image upload to map pin appearing (target ≤ 5 seconds on Colab/staging GPU).
- User flow completeness: citizen can submit a report, department staff can update its status, and citizen can see the updated status — verified by integration test.
- Staging reliability: platform available for login and submission during evaluation window (target $\geq 99\%$ uptime).

6.3 Pilot validation plan

- 10–15 test submissions using real photos of campus/city infrastructure issues.
- 3 simulated staff roles (team members) process the queue and update statuses.
- Record mAP50, per-class precision/recall, and end-to-end latency for each submission.
- Document failure cases (misdetections, wrong severity) honestly in the report limitations section.

7 Scalability

The proposition should scale in both theoretical and practical senses.

- Scalable (theoretically): stateless FastAPI backend enables horizontal scaling; PostgreSQL queries on GPS and status fields are indexed; image storage is decoupled from the database; ML inference can be offloaded to a GPU worker queue (e.g., Celery + Redis) without changing the API contract.

- Operationally deployable (practically): containerized backend (Docker); CI/CD to staging on every main merge; free-tier hosting sufficient for pilot; model weights versioned and swappable without redeployment.

8 Engine availability heuristic

A mature set of tools and services is available to implement this project without inventing infrastructure:

- YOLOv8 (Ultralytics): open-source, well-documented, fine-tunable on custom datasets with a single Python call.
- Roboflow Universe: free public labeled datasets for civic-issue classes; free annotation tool for custom labeling.
- Google Colab: free T4 GPU for training runs; sufficient for YOLOv8s on a dataset of 1,000–3,000 images.
- FastAPI + SQLAlchemy + PostgreSQL: standard Python web stack; welldocumented; background task support built in.
- React + react-leaflet + recharts: standard frontend stack; Leaflet is open-source with no API billing.
- GitHub Actions + Render: free-tier CI/CD and hosting for pilot scale.

We use these mature components to focus on system integration, workflow correctness, and measurable outcomes rather than building infrastructure from scratch.

9 Project scope and deliverables ...iteration-friendly

9.1 First iteration (Weeks 1–6)

- YOLOv8 fine-tuned model achieving $mAP50 \geq 0.65$ on held-out test set.
- FastAPI backend: image upload endpoint, ML inference integration, report storage in PostgreSQL.
- Citizen submission flow: photo upload + GPS capture + detection result display.
- Basic Leaflet map with real report pins from database.
- CI: linting + backend unit tests + frontend build on every push.
- CD: staging deployment via Render on every main merge.

9.2 Subsequent deliverables (Weeks 7–12)

- Duplicate detection and urgency scoring integrated into the report pipeline.
- Department staff dashboard: prioritized queue, status management, filters.
- Admin dashboard: city-wide map, resolution metrics, department performance charts.
- JWT authentication with role-based access (citizen / staff / admin).
- Integration testing: full end-to-end flow verified programmatically.

- Final model improvement run (YOLOv8s, 100 epochs) and per-class accuracy report.

10 Risks and mitigations

- Model accuracy on unseen images: mitigated by using diverse public datasets, augmenting with team-collected photos, and setting a confidence threshold (default 0.4) below which detections are flagged for human review rather than auto-accepted.
- Scope creep across six AI sub-features: mitigated by clearly marking which features are ML-driven (detection, severity) and which are rule-based (routing, duplicate check, urgency) — we build ML features first and add rule-based layers only once the core pipeline is validated.
- GPS inaccuracy or denial: mitigated by a manual map-pin fallback on every submission screen; duplicate detection radius is configurable to account for GPS drift.
- Colab session resets losing training progress: mitigated by saving all model weights and training runs directly to Google Drive, not Colab local storage.
- Free-tier hosting limits: mitigated by async inference (background tasks) so the API does not time out while the model runs; staging is sized for pilot load only.
- Data privacy: no personal data beyond an email address is collected; photo uploads are stored with a random filename, not linked to user identity in the public-facing map.

11 Summary

This project proposes a deployable, AI-powered civic reporting platform that automates the detection, severity estimation, duplicate filtering, and department routing of citizen-reported infrastructure issues. It meets the course criteria by leading with a validated ML pipeline (fine-tuned YOLOv8 computer vision), delivering working increments through sprint-based Agile development with CI/CD from the first week, and defining measurable evaluation criteria (mAP50, end-to-end latency, duplicate precision) rather than subjective claims. The project is deliberately scoped to distinguish ML-driven components from rule-based logic, and documents known limitations honestly. The result is a system that is technically substantial, socially relevant to the Indian urban context, and maps directly onto every unit of the UCS503 curriculum — requirements engineering, software design, Agile development, verification and validation, and project management.